In [6]:
```
import piplite
await piplite.install(['numpy'])
await piplite.install(['pandas'])
await piplite.install(['seaborn'])
```

```
---------------------------------------------------------------------------
ModuleNotFoundError                       Traceback (most recent call last)
Cell In[6], line 1
----> 1 import piplite
      2 await piplite.install(['numpy'])
      3 await piplite.install(['pandas'])

ModuleNotFoundError: No module named 'piplite'
```

In [7]:
```
# Pandas is a software library written for the Python programming language for d
import pandas as pd
# NumPy is a library for the Python programming language, adding support for lar
import numpy as np
# Matplotlib is a plotting library for python and pyplot gives us a MatLab like
import matplotlib.pyplot as plt
#Seaborn is a Python data visualization library based on matplotlib. It provides
import seaborn as sns
# Preprocessing allows us to standarsize our data
from sklearn import preprocessing
# Allows us to split our data into training and testing data
from sklearn.model_selection import train_test_split
# Allows us to test parameters of classification algorithms and find the best or
from sklearn.model_selection import GridSearchCV
# Logistic Regression classification algorithm
from sklearn.linear_model import LogisticRegression
# Support Vector Machine classification algorithm
from sklearn.svm import SVC
# Decision Tree classification algorithm
from sklearn.tree import DecisionTreeClassifier
# K Nearest Neighbors classification algorithm
from sklearn.neighbors import KNeighborsClassifier
```

In [8]:
```
def plot_confusion_matrix(y,y_predict):
    "this function plots the confusion matrix"
    from sklearn.metrics import confusion_matrix

    cm = confusion_matrix(y, y_predict)
    ax= plt.subplot()
    sns.heatmap(cm, annot=True, ax = ax); #annot=True to annotate cells
    ax.set_xlabel('Predicted labels')
    ax.set_ylabel('True labels')
    ax.set_title('Confusion Matrix');
    ax.xaxis.set_ticklabels(['did not land', 'land']); ax.yaxis.set_ticklabels(
    plt.show()
```

In [9]:
```
import pandas as pd

URL1 = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-D
data = pd.read_csv(URL1)
```

In [10]:
```
data.head()
```

Out[10]:

	FlightNumber	Date	BoosterVersion	PayloadMass	Orbit	LaunchSite	Outcom
0	1	2010-06-04	Falcon 9	6104.959412	LEO	CCAFS SLC 40	Nor Nor
1	2	2012-05-22	Falcon 9	525.000000	LEO	CCAFS SLC 40	Nor Nor
2	3	2013-03-01	Falcon 9	677.000000	ISS	CCAFS SLC 40	Nor Nor
3	4	2013-09-29	Falcon 9	500.000000	PO	VAFB SLC 4E	Fal Oce
4	5	2013-12-03	Falcon 9	3170.000000	GTO	CCAFS SLC 40	Nor Nor

In [11]:
```
import pandas as pd

URL2 = 'https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-D
X = pd.read_csv(URL2)

# Check the first few rows to confirm data loaded correctly
X.head()
```

Out[11]:

	FlightNumber	PayloadMass	Flights	Block	ReusedCount	Orbit_ES-L1	Orbit_GEO	Or
0	1.0	6104.959412	1.0	1.0	0.0	0.0	0.0	
1	2.0	525.000000	1.0	1.0	0.0	0.0	0.0	
2	3.0	677.000000	1.0	1.0	0.0	0.0	0.0	
3	4.0	500.000000	1.0	1.0	0.0	0.0	0.0	
4	5.0	3170.000000	1.0	1.0	0.0	0.0	0.0	

5 rows × 83 columns

In [12]:
```
# Check column names and data types
X.info()

# Check the first few rows
X.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 90 entries, 0 to 89
Data columns (total 83 columns):
 #   Column                                Non-Null Count  Dtype  
---  ------                                --------------  -----  
 0   FlightNumber                          90 non-null     float64
 1   PayloadMass                           90 non-null     float64
 2   Flights                               90 non-null     float64
 3   Block                                 90 non-null     float64
 4   ReusedCount                           90 non-null     float64
 5   Orbit_ES-L1                           90 non-null     float64
 6   Orbit_GEO                             90 non-null     float64
 7   Orbit_GTO                             90 non-null     float64
 8   Orbit_HEO                             90 non-null     float64
 9   Orbit_ISS                             90 non-null     float64
 10  Orbit_LEO                             90 non-null     float64
 11  Orbit_MEO                             90 non-null     float64
 12  Orbit_PO                              90 non-null     float64
 13  Orbit_SO                              90 non-null     float64
 14  Orbit_SSO                             90 non-null     float64
 15  Orbit_VLEO                            90 non-null     float64
 16  LaunchSite_CCAFS SLC 40               90 non-null     float64
 17  LaunchSite_KSC LC 39A                 90 non-null     float64
 18  LaunchSite_VAFB SLC 4E                90 non-null     float64
 19  LandingPad_5e9e3032383ecb267a34e7c7   90 non-null     float64
 20  LandingPad_5e9e3032383ecb554034e7c9   90 non-null     float64
 21  LandingPad_5e9e3032383ecb6bb234e7ca   90 non-null     float64
 22  LandingPad_5e9e3032383ecb761634e7cb   90 non-null     float64
 23  LandingPad_5e9e3033383ecbb9e534e7cc   90 non-null     float64
 24  Serial_B0003                          90 non-null     float64
 25  Serial_B0005                          90 non-null     float64
 26  Serial_B0007                          90 non-null     float64
 27  Serial_B1003                          90 non-null     float64
 28  Serial_B1004                          90 non-null     float64
 29  Serial_B1005                          90 non-null     float64
 30  Serial_B1006                          90 non-null     float64
 31  Serial_B1007                          90 non-null     float64
 32  Serial_B1008                          90 non-null     float64
 33  Serial_B1010                          90 non-null     float64
 34  Serial_B1011                          90 non-null     float64
 35  Serial_B1012                          90 non-null     float64
 36  Serial_B1013                          90 non-null     float64
 37  Serial_B1015                          90 non-null     float64
 38  Serial_B1016                          90 non-null     float64
 39  Serial_B1017                          90 non-null     float64
 40  Serial_B1018                          90 non-null     float64
 41  Serial_B1019                          90 non-null     float64
 42  Serial_B1020                          90 non-null     float64
 43  Serial_B1021                          90 non-null     float64
 44  Serial_B1022                          90 non-null     float64
 45  Serial_B1023                          90 non-null     float64
 46  Serial_B1025                          90 non-null     float64
 47  Serial_B1026                          90 non-null     float64
 48  Serial_B1028                          90 non-null     float64
 49  Serial_B1029                          90 non-null     float64
 50  Serial_B1030                          90 non-null     float64
```

```
 51  Serial_B1031                90 non-null     float64
 52  Serial_B1032                90 non-null     float64
 53  Serial_B1034                90 non-null     float64
 54  Serial_B1035                90 non-null     float64
 55  Serial_B1036                90 non-null     float64
 56  Serial_B1037                90 non-null     float64
 57  Serial_B1038                90 non-null     float64
 58  Serial_B1039                90 non-null     float64
 59  Serial_B1040                90 non-null     float64
 60  Serial_B1041                90 non-null     float64
 61  Serial_B1042                90 non-null     float64
 62  Serial_B1043                90 non-null     float64
 63  Serial_B1044                90 non-null     float64
 64  Serial_B1045                90 non-null     float64
 65  Serial_B1046                90 non-null     float64
 66  Serial_B1047                90 non-null     float64
 67  Serial_B1048                90 non-null     float64
 68  Serial_B1049                90 non-null     float64
 69  Serial_B1050                90 non-null     float64
 70  Serial_B1051                90 non-null     float64
 71  Serial_B1054                90 non-null     float64
 72  Serial_B1056                90 non-null     float64
 73  Serial_B1058                90 non-null     float64
 74  Serial_B1059                90 non-null     float64
 75  Serial_B1060                90 non-null     float64
 76  Serial_B1062                90 non-null     float64
 77  GridFins_False              90 non-null     float64
 78  GridFins_True               90 non-null     float64
 79  Reused_False                90 non-null     float64
 80  Reused_True                 90 non-null     float64
 81  Legs_False                  90 non-null     float64
 82  Legs_True                   90 non-null     float64
dtypes: float64(83)
memory usage: 58.5 KB
```

Out[12]:

	FlightNumber	PayloadMass	Flights	Block	ReusedCount	Orbit_ES-L1	Orbit_GEO	Or
0	1.0	6104.959412	1.0	1.0	0.0	0.0	0.0	
1	2.0	525.000000	1.0	1.0	0.0	0.0	0.0	
2	3.0	677.000000	1.0	1.0	0.0	0.0	0.0	
3	4.0	500.000000	1.0	1.0	0.0	0.0	0.0	
4	5.0	3170.000000	1.0	1.0	0.0	0.0	0.0	

5 rows × 83 columns

In [13]:
```
X.describe()
```

Out[13]:

	FlightNumber	PayloadMass	Flights	Block	ReusedCount	Orbit_ES-L1	Orl
count	90.000000	90.000000	90.000000	90.000000	90.000000	90.000000	90
mean	45.500000	6104.959412	1.788889	3.500000	1.655556	0.011111	0
std	26.124701	4694.671720	1.213172	1.595288	1.710254	0.105409	0
min	1.000000	350.000000	1.000000	1.000000	0.000000	0.000000	0
25%	23.250000	2510.750000	1.000000	2.000000	0.000000	0.000000	0
50%	45.500000	4701.500000	1.000000	4.000000	1.000000	0.000000	0
75%	67.750000	8912.750000	2.000000	5.000000	3.000000	0.000000	0
max	90.000000	15600.000000	6.000000	5.000000	5.000000	1.000000	1

8 rows × 83 columns

In [14]:
```
# Extract the 'Class' column as a Pandas Series, then convert it to a NumPy arra
Y = data['Class'].to_numpy()

# Check the result
print(type(Y))  # It should output: <class 'numpy.ndarray'>
```

```
<class 'numpy.ndarray'>
```

In [15]:
```
print(type(Y))
```

```
<class 'numpy.ndarray'>
```

In [16]:
```
# Import the StandardScaler
from sklearn.preprocessing import StandardScaler

# Create an instance of StandardScaler
transform = StandardScaler()

# Fit and transform the data, then assign it back to X
X = transform.fit_transform(X)

# Check the first few rows of the standardized data
print(X[:5])  # Just to display the first 5 rows after transformation
```

```
[[-1.71291154e+00 -1.94814463e-16 -6.53912840e-01 -1.57589457e+00
  -9.73440458e-01 -1.05999788e-01 -1.05999788e-01 -6.54653671e-01
  -1.05999788e-01 -5.51677284e-01  3.44342023e+00 -1.85695338e-01
  -3.33333333e-01 -1.05999788e-01 -2.42535625e-01 -4.29197538e-01
   7.97724035e-01 -5.68796459e-01 -4.10890702e-01 -4.10890702e-01
  -1.50755672e-01 -7.97724035e-01 -1.50755672e-01 -3.92232270e-01
   9.43398113e+00 -1.05999788e-01 -1.05999788e-01 -1.05999788e-01
  -1.05999788e-01 -1.05999788e-01 -1.05999788e-01 -1.05999788e-01
  -1.05999788e-01 -1.05999788e-01 -1.05999788e-01 -1.05999788e-01
  -1.05999788e-01 -1.05999788e-01 -1.05999788e-01 -1.05999788e-01
  -1.05999788e-01 -1.05999788e-01 -1.05999788e-01 -1.50755672e-01
  -1.05999788e-01 -1.05999788e-01 -1.05999788e-01 -1.05999788e-01
  -1.05999788e-01 -1.50755672e-01 -1.05999788e-01 -1.50755672e-01
  -1.50755672e-01 -1.05999788e-01 -1.50755672e-01 -1.50755672e-01
  -1.05999788e-01 -1.05999788e-01 -1.50755672e-01 -1.50755672e-01
  -1.50755672e-01 -1.05999788e-01 -1.05999788e-01 -1.05999788e-01
  -1.50755672e-01 -2.15665546e-01 -1.85695338e-01 -2.15665546e-01
  -2.67261242e-01 -1.05999788e-01 -2.42535625e-01 -1.05999788e-01
  -2.15665546e-01 -1.85695338e-01 -2.15665546e-01 -1.85695338e-01
  -1.05999788e-01  1.87082869e+00 -1.87082869e+00  8.35531692e-01
  -8.35531692e-01  1.93309133e+00 -1.93309133e+00]
 [-1.67441914e+00 -1.19523159e+00 -6.53912840e-01 -1.57589457e+00
  -9.73440458e-01 -1.05999788e-01 -1.05999788e-01 -6.54653671e-01
  -1.05999788e-01 -5.51677284e-01  3.44342023e+00 -1.85695338e-01
  -3.33333333e-01 -1.05999788e-01 -2.42535625e-01 -4.29197538e-01
   7.97724035e-01 -5.68796459e-01 -4.10890702e-01 -4.10890702e-01
  -1.50755672e-01 -7.97724035e-01 -1.50755672e-01 -3.92232270e-01
  -1.05999788e-01  9.43398113e+00 -1.05999788e-01 -1.05999788e-01
  -1.05999788e-01 -1.05999788e-01 -1.05999788e-01 -1.05999788e-01
  -1.05999788e-01 -1.05999788e-01 -1.05999788e-01 -1.05999788e-01
  -1.05999788e-01 -1.05999788e-01 -1.05999788e-01 -1.50755672e-01
  -1.05999788e-01 -1.05999788e-01 -1.05999788e-01 -1.05999788e-01
  -1.05999788e-01 -1.50755672e-01 -1.05999788e-01 -1.50755672e-01
  -1.50755672e-01 -1.05999788e-01 -1.50755672e-01 -1.50755672e-01
  -1.05999788e-01 -1.05999788e-01 -1.50755672e-01 -1.50755672e-01
  -1.50755672e-01 -1.05999788e-01 -1.05999788e-01 -1.05999788e-01
  -1.50755672e-01 -2.15665546e-01 -1.85695338e-01 -2.15665546e-01
  -2.67261242e-01 -1.05999788e-01 -2.42535625e-01 -1.05999788e-01
  -2.15665546e-01 -1.85695338e-01 -2.15665546e-01 -1.85695338e-01
  -1.05999788e-01  1.87082869e+00 -1.87082869e+00  8.35531692e-01
  -8.35531692e-01  1.93309133e+00 -1.93309133e+00]
 [-1.63592675e+00 -1.16267307e+00 -6.53912840e-01 -1.57589457e+00
  -9.73440458e-01 -1.05999788e-01 -1.05999788e-01 -6.54653671e-01
  -1.05999788e-01  1.81265393e+00 -2.90408935e-01 -1.85695338e-01
  -3.33333333e-01 -1.05999788e-01 -2.42535625e-01 -4.29197538e-01
   7.97724035e-01 -5.68796459e-01 -4.10890702e-01 -4.10890702e-01
  -1.50755672e-01 -7.97724035e-01 -1.50755672e-01 -3.92232270e-01
  -1.05999788e-01 -1.05999788e-01  9.43398113e+00 -1.05999788e-01
  -1.05999788e-01 -1.05999788e-01 -1.05999788e-01 -1.05999788e-01
  -1.05999788e-01 -1.05999788e-01 -1.05999788e-01 -1.05999788e-01
  -1.05999788e-01 -1.05999788e-01 -1.05999788e-01 -1.05999788e-01
  -1.05999788e-01 -1.05999788e-01 -1.05999788e-01 -1.50755672e-01
  -1.05999788e-01 -1.05999788e-01 -1.05999788e-01 -1.05999788e-01
  -1.05999788e-01 -1.50755672e-01 -1.05999788e-01 -1.50755672e-01
  -1.50755672e-01 -1.05999788e-01 -1.50755672e-01 -1.50755672e-01
```

```
 -1.05999788e-01 -1.05999788e-01 -1.50755672e-01 -1.50755672e-01
 -1.50755672e-01 -1.05999788e-01 -1.05999788e-01 -1.05999788e-01
 -1.50755672e-01 -2.15665546e-01 -1.85695338e-01 -2.15665546e-01
 -2.67261242e-01 -1.05999788e-01 -2.42535625e-01 -1.05999788e-01
 -2.15665546e-01 -1.85695338e-01 -2.15665546e-01 -1.85695338e-01
 -1.05999788e-01  1.87082869e+00 -1.87082869e+00  8.35531692e-01
 -8.35531692e-01  1.93309133e+00 -1.93309133e+00]
[-1.59743435e+00 -1.20058661e+00 -6.53912840e-01 -1.57589457e+00
 -9.73440458e-01 -1.05999788e-01 -1.05999788e-01 -6.54653671e-01
 -1.05999788e-01 -5.51677284e-01 -2.90408935e-01 -1.85695338e-01
  3.00000000e+00 -1.05999788e-01 -2.42535625e-01 -4.29197538e-01
 -1.25356634e+00 -5.68796459e-01  2.43373723e+00 -4.10890702e-01
 -1.50755672e-01 -7.97724035e-01 -1.50755672e-01 -3.92232270e-01
 -1.05999788e-01 -1.05999788e-01 -1.05999788e-01  9.43398113e+00
 -1.05999788e-01 -1.05999788e-01 -1.05999788e-01 -1.05999788e-01
 -1.05999788e-01 -1.05999788e-01 -1.05999788e-01 -1.05999788e-01
 -1.05999788e-01 -1.05999788e-01 -1.05999788e-01 -1.05999788e-01
 -1.05999788e-01 -1.05999788e-01 -1.05999788e-01 -1.50755672e-01
 -1.05999788e-01 -1.05999788e-01 -1.05999788e-01 -1.05999788e-01
 -1.05999788e-01 -1.50755672e-01 -1.05999788e-01 -1.50755672e-01
 -1.50755672e-01 -1.05999788e-01 -1.50755672e-01 -1.50755672e-01
 -1.05999788e-01 -1.05999788e-01 -1.50755672e-01 -1.50755672e-01
 -1.50755672e-01 -1.05999788e-01 -1.05999788e-01 -1.05999788e-01
 -1.50755672e-01 -2.15665546e-01 -1.85695338e-01 -2.15665546e-01
 -2.67261242e-01 -1.05999788e-01 -2.42535625e-01 -1.05999788e-01
 -2.15665546e-01 -1.85695338e-01 -2.15665546e-01 -1.85695338e-01
 -1.05999788e-01  1.87082869e+00 -1.87082869e+00  8.35531692e-01
 -8.35531692e-01  1.93309133e+00 -1.93309133e+00]
[-1.55894196e+00 -6.28670558e-01 -6.53912840e-01 -1.57589457e+00
 -9.73440458e-01 -1.05999788e-01 -1.05999788e-01  1.52752523e+00
 -1.05999788e-01 -5.51677284e-01 -2.90408935e-01 -1.85695338e-01
 -3.33333333e-01 -1.05999788e-01 -2.42535625e-01 -4.29197538e-01
  7.97724035e-01 -5.68796459e-01 -4.10890702e-01 -4.10890702e-01
 -1.50755672e-01 -7.97724035e-01 -1.50755672e-01 -3.92232270e-01
 -1.05999788e-01 -1.05999788e-01 -1.05999788e-01 -1.05999788e-01
  9.43398113e+00 -1.05999788e-01 -1.05999788e-01 -1.05999788e-01
 -1.05999788e-01 -1.05999788e-01 -1.05999788e-01 -1.05999788e-01
 -1.05999788e-01 -1.05999788e-01 -1.05999788e-01 -1.05999788e-01
 -1.05999788e-01 -1.05999788e-01 -1.05999788e-01 -1.50755672e-01
 -1.05999788e-01 -1.05999788e-01 -1.05999788e-01 -1.05999788e-01
 -1.05999788e-01 -1.50755672e-01 -1.05999788e-01 -1.50755672e-01
 -1.50755672e-01 -1.05999788e-01 -1.50755672e-01 -1.50755672e-01
 -1.05999788e-01 -1.05999788e-01 -1.50755672e-01 -1.50755672e-01
 -1.50755672e-01 -1.05999788e-01 -1.05999788e-01 -1.05999788e-01
 -1.50755672e-01 -2.15665546e-01 -1.85695338e-01 -2.15665546e-01
 -2.67261242e-01 -1.05999788e-01 -2.42535625e-01 -1.05999788e-01
 -2.15665546e-01 -1.85695338e-01 -2.15665546e-01 -1.85695338e-01
 -1.05999788e-01  1.87082869e+00 -1.87082869e+00  8.35531692e-01
 -8.35531692e-01  1.93309133e+00 -1.93309133e+00]]
```

In [17]:

```
from sklearn.model_selection import train_test_split

# Split the data into training and test sets with 20% data for testing
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_

# Check the shapes of the resulting sets to confirm the split
```

```
print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("Y_train shape:", Y_train.shape)
print("Y_test shape:", Y_test.shape)
```

```
X_train shape: (72, 83)
X_test shape: (18, 83)
Y_train shape: (72,)
Y_test shape: (18,)
```

In [18]:
```
Y_test.shape
```

Out[18]: (18,)

In [19]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV

# Step 1: Create a Logistic Regression object
logreg = LogisticRegression()

# Step 2: Define the parameter grid
parameters = {
    'C': [0.01, 0.1, 1],  # Regularization strength
    'penalty': ['l2'],     # Type of regularization (L2)
    'solver': ['lbfgs']    # Optimization algorithm
}

# Step 3: Set up GridSearchCV with 10-fold cross-validation
logreg_cv = GridSearchCV(logreg, parameters, cv=10)

# Step 4: Fit the model to find the best parameters
logreg_cv.fit(X_train, Y_train)

# Step 5: Output the best parameters found
print("Best parameters found: ", logreg_cv.best_params_)
```

```
Best parameters found:  {'C': 0.01, 'penalty': 'l2', 'solver': 'lbfgs'}
```

In [20]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV

# Define the Logistic Regression model
lr = LogisticRegression()

# Define the parameter grid for tuning
parameters = {
    'C': [0.01, 0.1, 1],      # Regularization strengths
    'penalty': ['l2'],        # L2 regularization (ridge)
    'solver': ['lbfgs']       # Optimization algorithm (lbfgs)
}

# Create a GridSearchCV object with 10-fold cross-validation
logreg_cv = GridSearchCV(lr, parameters, cv=10)

# Fit the model to the training data
logreg_cv.fit(X_train, Y_train)
```

```
# Output the best parameters found by GridSearchCV
print("Best parameters found: ", logreg_cv.best_params_)
```

```
Best parameters found:  {'C': 0.01, 'penalty': 'l2', 'solver': 'lbfgs'}
```

In [21]:
```
# Output the best parameters found by GridSearchCV
print("Best parameters found: ", logreg_cv.best_params_)

# Output the best score (accuracy) obtained on the validation data
print("Best cross-validation accuracy: ", logreg_cv.best_score_)
```

```
Best parameters found:  {'C': 0.01, 'penalty': 'l2', 'solver': 'lbfgs'}
Best cross-validation accuracy:  0.8464285714285713
```

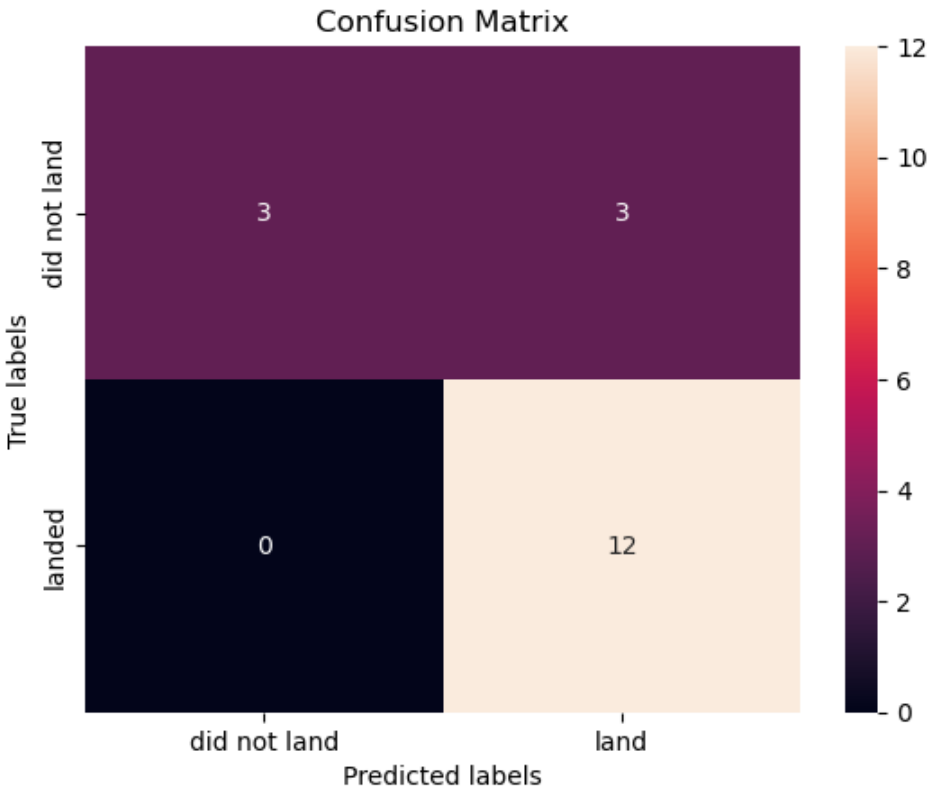
In [22]:
```
print("tuned hyperparameters (best parameters):", logreg_cv.best_params_)
print("accuracy:", logreg_cv.best_score_)
```

```
tuned hyperparameters (best parameters): {'C': 0.01, 'penalty': 'l2', 'solver':
'lbfgs'}
accuracy: 0.8464285714285713
```

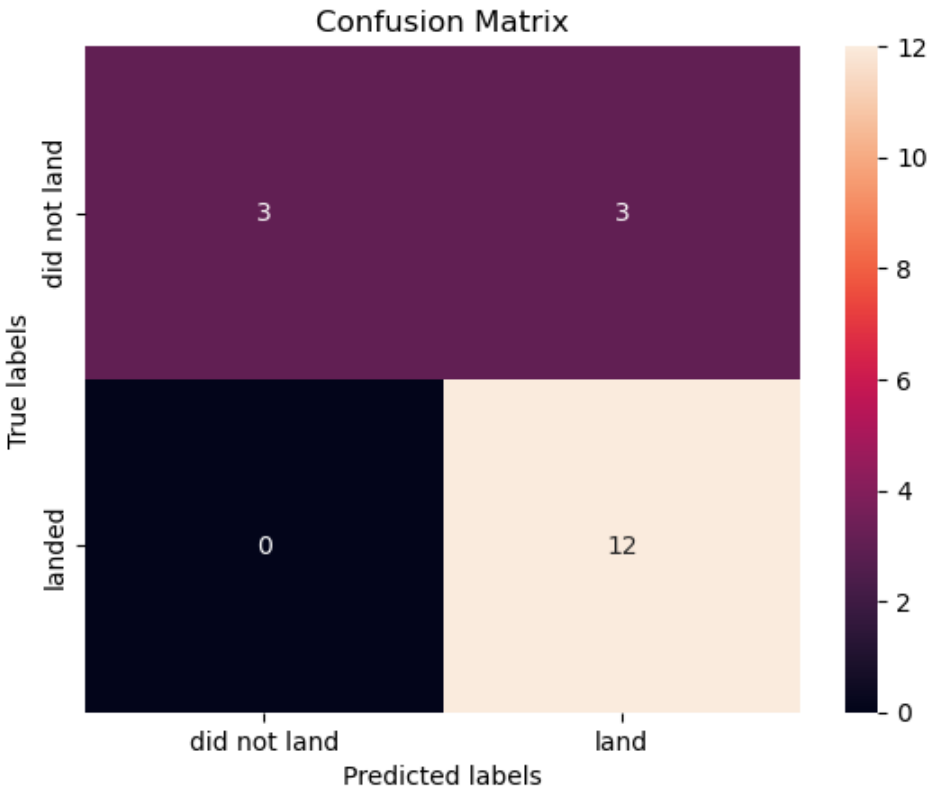
In [23]:
```
# Calculate the accuracy on the test data using the score method
test_accuracy = logreg_cv.score(X_test, Y_test)

# Output the accuracy on the test data
print("Accuracy on the test data: ", test_accuracy)
```

```
Accuracy on the test data:  0.8333333333333334
```

In [24]:
```
# Make predictions on the test set
yhat = logreg_cv.predict(X_test)

# Plot the confusion matrix
plot_confusion_matrix(Y_test, yhat)
```



In [25]:
```
# Make predictions on the test set
yhat = logreg_cv.predict(X_test)

# Plot the confusion matrix
plot_confusion_matrix(Y_test, yhat)
```



In [26]:
```
def plot_confusion_matrix(y, y_predict):
    from sklearn.metrics import confusion_matrix
    cm = confusion_matrix(y, y_predict)
    ax = plt.subplot()
    sns.heatmap(cm, annot=True, ax=ax, fmt='g')  # g is for integer format
    ax.set_xlabel('Predicted labels')
    ax.set_ylabel('True labels')
    ax.set_title('Confusion Matrix')
    ax.xaxis.set_ticklabels(['did not land', 'landed'])
    ax.yaxis.set_ticklabels(['did not land', 'landed'])
    plt.show()
```

In [27]:
```
parameters = {'kernel':('linear', 'rbf','poly','rbf', 'sigmoid'),
              'C': np.logspace(-3, 3, 5),
              'gamma':np.logspace(-3, 3, 5)}
svm = SVC()
```

In [28]:
```
# Import the necessary libraries
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV
import numpy as np

# Create the SVM object
svm = SVC()

# Define the parameter grid for the GridSearchCV
parameters = {
    'kernel': ('linear', 'rbf', 'poly', 'sigmoid'),  # Different kernels to cho
```

```
    'C': np.logspace(-3, 3, 5),  # Regularization parameter C from 0.001 to 1000
    'gamma': np.logspace(-3, 3, 5)  # Kernel coefficient gamma from 0.001 to 100
}

# Set up the GridSearchCV object with 10-fold cross-validation
svm_cv = GridSearchCV(svm, parameters, cv=10, n_jobs=-1)

# Fit the model to the training data (X_train, Y_train)
svm_cv.fit(X_train, Y_train)

# Output the best parameters and the best score found during the GridSearchCV
print("Best parameters found: ", svm_cv.best_params_)
print("Best cross-validation accuracy: ", svm_cv.best_score_)
```

```
Best parameters found:  {'C': 1.0, 'gamma': 0.03162277660168379, 'kernel': 'sigm
oid'}
Best cross-validation accuracy:  0.8482142857142856
```

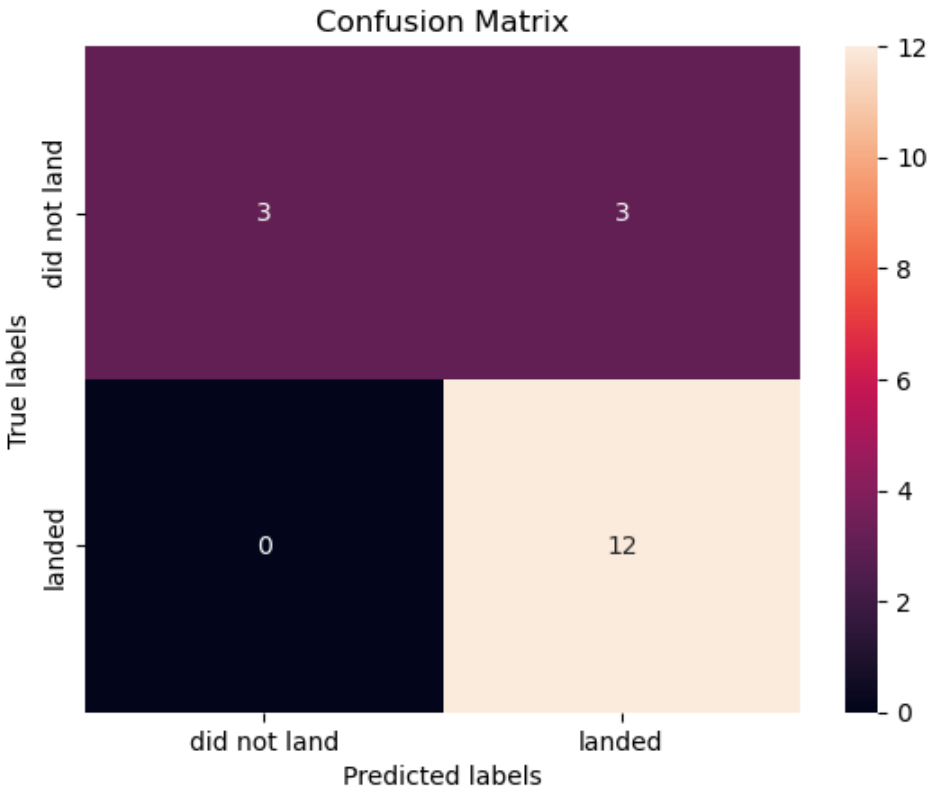
In [29]:
```
print("tuned hpyerparameters :(best parameters) ",svm_cv.best_params_)
print("accuracy :",svm_cv.best_score_)
```

```
tuned hpyerparameters :(best parameters)  {'C': 1.0, 'gamma': 0.0316227766016837
9, 'kernel': 'sigmoid'}
accuracy : 0.8482142857142856
```

In [30]:
```
# Calculate the accuracy of the best model on the test data
test_accuracy = svm_cv.best_estimator_.score(X_test, Y_test)

# Output the accuracy
print("Accuracy on the test data: ", test_accuracy)
```

```
Accuracy on the test data:  0.8333333333333334
```

In [31]:
```
yhat=svm_cv.predict(X_test)
plot_confusion_matrix(Y_test,yhat)
```



In [32]:
```
# Import necessary libraries
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV

# Create the Decision Tree classifier object
tree = DecisionTreeClassifier()

# Define the parameter grid for the GridSearchCV
parameters = {
    'criterion': ['gini', 'entropy'],  # Criteria for splitting (Gini impurity 
    'splitter': ['best', 'random'],    # Strategy for splitting nodes
    'max_depth': [2*n for n in range(1, 10)],  # Maximum depth of the tree
    'max_features': ['auto', 'sqrt'],   # Number of features to consider for spl
    'min_samples_leaf': [1, 2, 4],      # Minimum samples required to be at a le
    'min_samples_split': [2, 5, 10]     # Minimum samples required to split an 
}

# Set up the GridSearchCV object with 10-fold cross-validation
tree_cv = GridSearchCV(tree, parameters, cv=10, n_jobs=-1)

# Fit the model to the training data (X_train, Y_train)
tree_cv.fit(X_train, Y_train)

# Output the best parameters and best score
print("Best parameters found: ", tree_cv.best_params_)
print("Best cross-validation accuracy: ", tree_cv.best_score_)
```

```
Best parameters found:  {'criterion': 'gini', 'max_depth': 8, 'max_features': 's
qrt', 'min_samples_leaf': 1, 'min_samples_split': 5, 'splitter': 'random'}
Best cross-validation accuracy:  0.8642857142857142
```

```
C:\Users\Michael\anaconda3\Lib\site-packages\sklearn\model_selection\_validatio
n.py:528: FitFailedWarning:
3240 fits failed out of a total of 6480.
The score on these train-test partitions for these parameters will be set to na
n.
If these failures are not expected, you can try to debug them by setting error_s
core='raise'.

Below are more details about the failures:
--------------------------------------------------------------------------------
2469 fits failed with the following error:
Traceback (most recent call last):
  File "C:\Users\Michael\anaconda3\Lib\site-packages\sklearn\model_selection\_va
lidation.py", line 866, in _fit_and_score
    estimator.fit(X_train, y_train, **fit_params)
  File "C:\Users\Michael\anaconda3\Lib\site-packages\sklearn\base.py", line 1382
, in wrapper
    estimator._validate_params()
  File "C:\Users\Michael\anaconda3\Lib\site-packages\sklearn\base.py", line 436,
in _validate_params
    validate_parameter_constraints(
  File "C:\Users\Michael\anaconda3\Lib\site-packages\sklearn\utils\_param_valida
tion.py", line 98, in validate_parameter_constraints
    raise InvalidParameterError(
sklearn.utils._param_validation.InvalidParameterError: The 'max_features' parame
ter of DecisionTreeClassifier must be an int in the range [1, inf), a float in t
he range (0.0, 1.0], a str among {'sqrt', 'log2'} or None. Got 'auto' instead.

--------------------------------------------------------------------------------
771 fits failed with the following error:
Traceback (most recent call last):
  File "C:\Users\Michael\anaconda3\Lib\site-packages\sklearn\model_selection\_va
lidation.py", line 866, in _fit_and_score
    estimator.fit(X_train, y_train, **fit_params)
  File "C:\Users\Michael\anaconda3\Lib\site-packages\sklearn\base.py", line 1382
, in wrapper
    estimator._validate_params()
  File "C:\Users\Michael\anaconda3\Lib\site-packages\sklearn\base.py", line 436,
in _validate_params
    validate_parameter_constraints(
  File "C:\Users\Michael\anaconda3\Lib\site-packages\sklearn\utils\_param_valida
tion.py", line 98, in validate_parameter_constraints
    raise InvalidParameterError(
sklearn.utils._param_validation.InvalidParameterError: The 'max_features' parame
ter of DecisionTreeClassifier must be an int in the range [1, inf), a float in t
he range (0.0, 1.0], a str among {'log2', 'sqrt'} or None. Got 'auto' instead.

  warnings.warn(some_fits_failed_message, FitFailedWarning)
C:\Users\Michael\anaconda3\Lib\site-packages\sklearn\model_selection\_search.p
y:1108: UserWarning: One or more of the test scores are non-finite: [       nan
nan        nan        nan        nan        nan
        nan        nan        nan        nan        nan        nan
        nan        nan        nan        nan        nan        nan
 0.75357143 0.78214286 0.73214286 0.77678571 0.83214286 0.79107143
 0.76607143 0.77857143 0.72321429 0.73214286 0.80535714 0.81964286
 0.74821429 0.80357143 0.77678571 0.76607143 0.78928571 0.76428571
```

```
       nan        nan        nan        nan        nan        nan
       nan        nan        nan        nan        nan        nan
       nan        nan        nan        nan        nan        nan
0.84642857 0.84642857 0.85714286 0.80714286 0.80357143 0.78214286
0.775      0.76071429 0.80714286 0.84642857 0.81964286 0.7625
0.74642857 0.775      0.775      0.78928571 0.76428571 0.79464286
       nan        nan        nan        nan        nan        nan
       nan        nan        nan        nan        nan        nan
       nan        nan        nan        nan        nan        nan
0.77857143 0.81964286 0.74821429 0.80357143 0.67857143 0.80357143
0.79107143 0.76785714 0.7625     0.74642857 0.76071429 0.83214286
0.68035714 0.83571429 0.80535714 0.66428571 0.81785714 0.7625
       nan        nan        nan        nan        nan        nan
       nan        nan        nan        nan        nan        nan
       nan        nan        nan        nan        nan        nan
0.75178571 0.77678571 0.81785714 0.86428571 0.84642857 0.81785714
0.70535714 0.73214286 0.72142857 0.775      0.79285714 0.79107143
0.84642857 0.83214286 0.7625     0.81964286 0.79285714 0.725
       nan        nan        nan        nan        nan        nan
       nan        nan        nan        nan        nan        nan
       nan        nan        nan        nan        nan        nan
0.81964286 0.80535714 0.74821429 0.75178571 0.80357143 0.83571429
0.80357143 0.7625     0.7625     0.79285714 0.72321429 0.82142857
0.74642857 0.71071429 0.79107143 0.79107143 0.71785714 0.74821429
       nan        nan        nan        nan        nan        nan
       nan        nan        nan        nan        nan        nan
       nan        nan        nan        nan        nan        nan
0.83392857 0.71071429 0.80357143 0.7875     0.75357143 0.79642857
0.7625     0.77678571 0.7625     0.78928571 0.81785714 0.79107143
0.73214286 0.77678571 0.73571429 0.83392857 0.78928571 0.81964286
       nan        nan        nan        nan        nan        nan
       nan        nan        nan        nan        nan        nan
       nan        nan        nan        nan        nan        nan
0.7375     0.75       0.75       0.76428571 0.77678571 0.80357143
0.77678571 0.775      0.70892857 0.72321429 0.76071429 0.78928571
0.71964286 0.81071429 0.73214286 0.81785714 0.81785714 0.78214286
       nan        nan        nan        nan        nan        nan
       nan        nan        nan        nan        nan        nan
       nan        nan        nan        nan        nan        nan
0.80357143 0.86071429 0.75       0.79285714 0.79107143 0.77857143
0.81428571 0.83214286 0.77857143 0.73392857 0.78928571 0.81607143
0.77678571 0.81071429 0.77857143 0.77857143 0.73214286 0.82142857
       nan        nan        nan        nan        nan        nan
       nan        nan        nan        nan        nan        nan
       nan        nan        nan        nan        nan        nan
0.77857143 0.79107143 0.73928571 0.77678571 0.71964286 0.80714286
0.82142857 0.76607143 0.79464286 0.77678571 0.77678571 0.80535714
0.75178571 0.83214286 0.73571429 0.76607143 0.71964286 0.81964286
       nan        nan        nan        nan        nan        nan
       nan        nan        nan        nan        nan        nan
       nan        nan        nan        nan        nan        nan
0.81964286 0.76785714 0.76607143 0.7625     0.75178571 0.74642857
0.79464286 0.75178571 0.80357143 0.69642857 0.80178571 0.68214286
0.79107143 0.77678571 0.76607143 0.76607143 0.81785714 0.86071429
       nan        nan        nan        nan        nan        nan
       nan        nan        nan        nan        nan        nan
```

```
       nan        nan        nan        nan        nan        nan
0.83214286 0.86071429 0.80178571 0.84642857 0.86428571 0.79285714
0.775      0.78928571 0.80535714 0.81785714 0.80535714 0.74821429
0.78035714 0.78214286 0.70535714 0.83214286 0.83214286 0.81607143
       nan        nan        nan        nan        nan        nan
       nan        nan        nan        nan        nan        nan
       nan        nan        nan        nan        nan        nan
0.81785714 0.82321429 0.82857143 0.83392857 0.75       0.83214286
0.75357143 0.84642857 0.80535714 0.80535714 0.72321429 0.72142857
0.79107143 0.80357143 0.76607143 0.7375     0.83214286 0.72142857
       nan        nan        nan        nan        nan        nan
       nan        nan        nan        nan        nan        nan
       nan        nan        nan        nan        nan        nan
0.77857143 0.76607143 0.81607143 0.81785714 0.80535714 0.84821429
0.77857143 0.80535714 0.82321429 0.83392857 0.76607143 0.78928571
0.84821429 0.83214286 0.7625     0.79464286 0.80357143 0.83392857
       nan        nan        nan        nan        nan        nan
       nan        nan        nan        nan        nan        nan
       nan        nan        nan        nan        nan        nan
0.83035714 0.83214286 0.77678571 0.79285714 0.75       0.81785714
0.77857143 0.775      0.74464286 0.80535714 0.76071429 0.80535714
0.775      0.84821429 0.81785714 0.70535714 0.74821429 0.81964286
       nan        nan        nan        nan        nan        nan
       nan        nan        nan        nan        nan        nan
       nan        nan        nan        nan        nan        nan
0.83214286 0.72142857 0.73571429 0.74821429 0.72678571 0.84642857
0.73571429 0.84642857 0.72142857 0.75       0.80357143 0.84821429
0.80357143 0.65535714 0.68214286 0.77857143 0.78928571 0.82142857
       nan        nan        nan        nan        nan        nan
       nan        nan        nan        nan        nan        nan
       nan        nan        nan        nan        nan        nan
0.71964286 0.775      0.7375     0.72142857 0.80535714 0.79107143
0.81964286 0.80714286 0.74642857 0.76071429 0.6625     0.70535714
0.83214286 0.79107143 0.80178571 0.775      0.77857143 0.84821429
       nan        nan        nan        nan        nan        nan
       nan        nan        nan        nan        nan        nan
       nan        nan        nan        nan        nan        nan
0.80714286 0.76428571 0.79107143 0.77678571 0.75       0.79107143
0.76428571 0.79107143 0.76071429 0.73571429 0.64821429 0.83392857
0.69642857 0.79285714 0.80357143 0.81964286 0.81964286 0.74821429
       nan        nan        nan        nan        nan        nan
       nan        nan        nan        nan        nan        nan
       nan        nan        nan        nan        nan        nan
0.77857143 0.72142857 0.74642857 0.76428571 0.73571429 0.75
0.775      0.77678571 0.76607143 0.82142857 0.77857143 0.77857143
0.76071429 0.79107143 0.67857143 0.80535714 0.78928571 0.74821429]
 warnings.warn(
```

In [32]:
```
print("tuned hpyerparameters :(best parameters) ",tree_cv.best_params_)
print("accuracy :",tree_cv.best_score_)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[32], line 1
----> 1 print("tuned hpyerparameters :(best parameters) ",tree_cv.best_params_)
      2 print("accuracy :",tree_cv.best_score_)

NameError: name 'tree_cv' is not defined
```

In [34]:
```
# Calculate the accuracy on the test data
test_accuracy = tree_cv.score(X_test, Y_test)

# Print the accuracy on the test data
print("Accuracy on the test data: ", test_accuracy)
```

```
Accuracy on the test data:  0.8333333333333334
```

In [35]:
```
# Import necessary libraries
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import GridSearchCV

# Create the KNN classifier object
KNN = KNeighborsClassifier()

# Define the parameter grid for the GridSearchCV
parameters = {
    'n_neighbors': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],    # Number of neighbors to
    'algorithm': ['auto', 'ball_tree', 'kd_tree', 'brute'],  # Algorithm for co
    'p': [1, 2]    # Distance metric (1 for Manhattan distance, 2 for Euclidean
}

# Set up the GridSearchCV object with 10-fold cross-validation
knn_cv = GridSearchCV(KNN, parameters, cv=10, n_jobs=-1)

# Fit the model to the training data (X_train, Y_train)
knn_cv.fit(X_train, Y_train)

# Output the best parameters and best score
print("Best parameters found: ", knn_cv.best_params_)
print("Best cross-validation accuracy: ", knn_cv.best_score_)
```

```
Best parameters found:  {'algorithm': 'auto', 'n_neighbors': 10, 'p': 1}
Best cross-validation accuracy:  0.8482142857142858
```

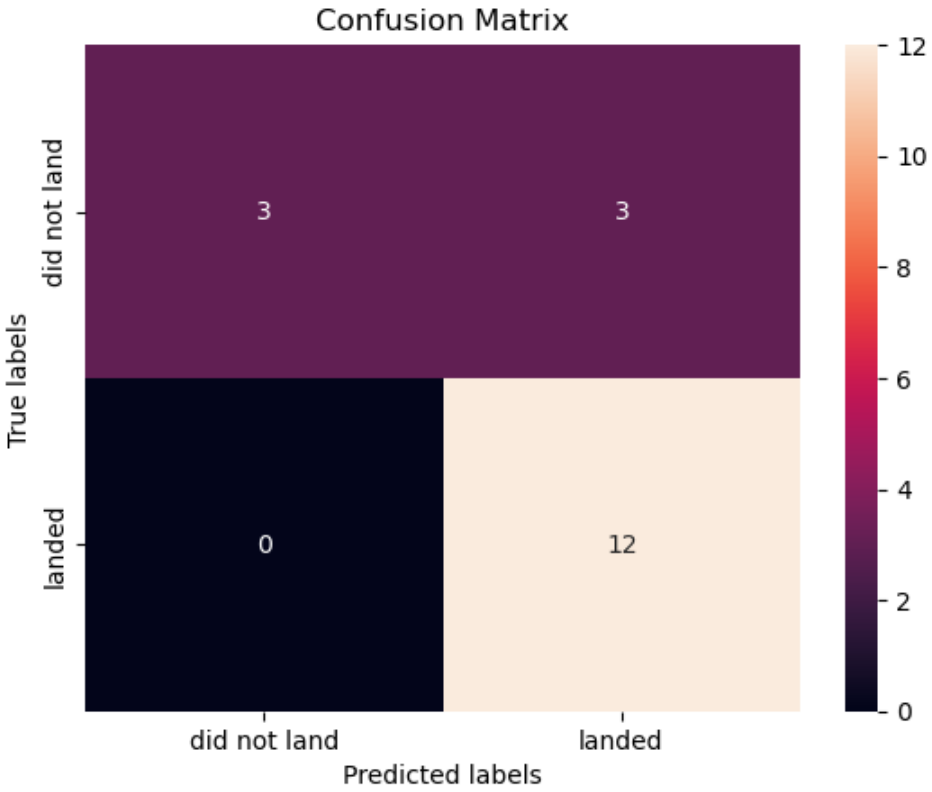
In [36]:
```
print("tuned hpyerparameters :(best parameters) ",knn_cv.best_params_)
print("accuracy :",knn_cv.best_score_)
```

```
tuned hpyerparameters :(best parameters)  {'algorithm': 'auto', 'n_neighbors': 1
0, 'p': 1}
accuracy : 0.8482142857142858
```

In [37]:
```
# Calculate the accuracy of the best KNN model on the test data
test_accuracy_knn = knn_cv.score(X_test, Y_test)

# Print the accuracy on the test data
print("Accuracy on the test data: ", test_accuracy_knn)
```

```
Accuracy on the test data:  0.8333333333333334
```

In [38]:
```
yhat = knn_cv.predict(X_test)
plot_confusion_matrix(Y_test,yhat)
```



In [39]:
```
# Logistic Regression accuracy (from earlier)
logreg_accuracy = logreg_cv.score(X_test, Y_test)

# Support Vector Machine accuracy (from earlier)
svm_accuracy = svm_cv.score(X_test, Y_test)

# Decision Tree accuracy (from earlier)
tree_accuracy = tree_cv.score(X_test, Y_test)

# K Nearest Neighbors accuracy (from earlier)
knn_accuracy = knn_cv.score(X_test, Y_test)

# Print all the accuracies
print("Logistic Regression Accuracy on Test Data: ", logreg_accuracy)
print("SVM Accuracy on Test Data: ", svm_accuracy)
print("Decision Tree Accuracy on Test Data: ", tree_accuracy)
print("KNN Accuracy on Test Data: ", knn_accuracy)

# Compare the best performing model
accuracies = {'Logistic Regression': logreg_accuracy,
              'SVM': svm_accuracy,
              'Decision Tree': tree_accuracy,
              'KNN': knn_accuracy}

best_model = max(accuracies, key=accuracies.get)
```

```
print("\nThe best performing model on the test data is: ", best_model)
```

```
Logistic Regression Accuracy on Test Data:  0.8333333333333334
SVM Accuracy on Test Data:  0.8333333333333334
Decision Tree Accuracy on Test Data:  0.8333333333333334
KNN Accuracy on Test Data:  0.8333333333333334

The best performing model on the test data is:  Logistic Regression
```

In [40]:
```
# Number of records in the test sample
len(X_test)
```

Out[40]: 18

In [41]:
```
X_test.shape[0]  # This gives the number of rows (records) in the test sample
```

Out[41]: 18

In [42]:
```
print("Best kernel for SVM:", svm_cv.best_params_['kernel'])
```

```
Best kernel for SVM: sigmoid
```

In [43]:
```
# Accuracy of the decision tree classifier on the test data
print("Decision Tree Accuracy on Test Data:", tree_cv.score(X_test, Y_test))
```

```
Decision Tree Accuracy on Test Data: 0.8333333333333334
```

In [1]:
```
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

def compare_classifiers(X_train, X_test, y_train, y_test, classifiers):
    """
    Trains multiple classifiers and plots confusion matrices for each.

    Parameters:
    - X_train, X_test, y_train, y_test: Data splits
    - classifiers: Dictionary of model name → classifier object
    """
    for name, clf in classifiers.items():
        clf.fit(X_train, y_train)
        y_pred = clf.predict(X_test)

        acc = accuracy_score(y_test, y_pred)
        f1 = f1_score(y_test, y_pred)

        # Plot confusion matrix
        cm = confusion_matrix(y_test, y_pred)
        plt.figure(figsize=(5, 4))
        sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False)
        plt.title(f'{name} - Confusion Matrix\nAccuracy: {acc:.2f}, F1-Score: {
        plt.xlabel('Predicted')
        plt.ylabel('Actual')
        plt.xticks(ticks=[0.5, 1.5], labels=['Did Not Land', 'Landed'])
        plt.yticks(ticks=[0.5, 1.5], labels=['Did Not Land', 'Landed'], rotatio
        plt.tight_layout()
        plt.show()
```

In [2]:
```
classifiers = {
    "Logistic Regression": LogisticRegression(),
    "SVM": SVC(),
    "Decision Tree": DecisionTreeClassifier(),
    "KNN": KNeighborsClassifier()
}

compare_classifiers(X_train, X_test, y_train, y_test, classifiers)
```

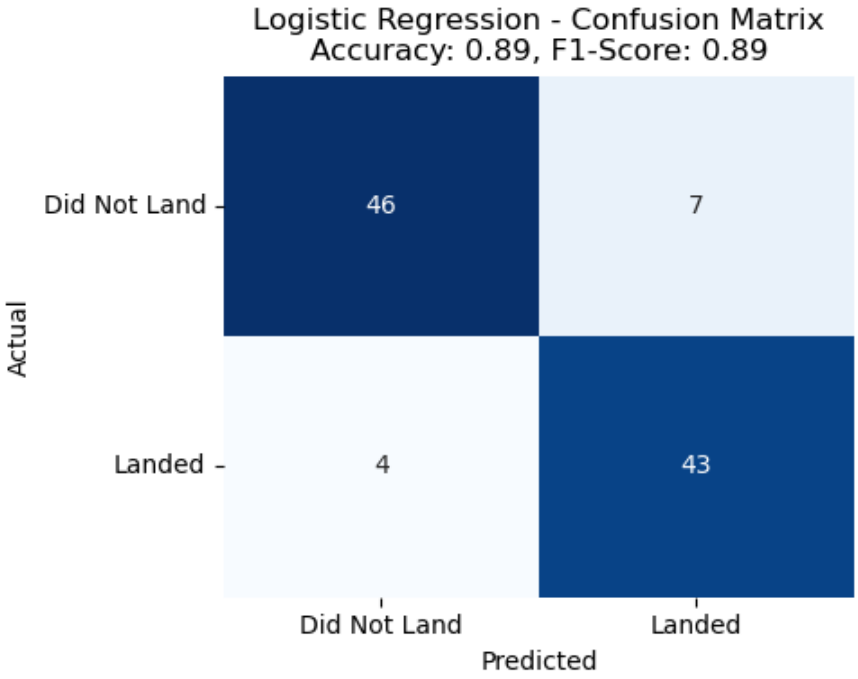
```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[2], line 2
      1 classifiers = {
----> 2     "Logistic Regression": LogisticRegression(),
      3     "SVM": SVC(),
      4     "Decision Tree": DecisionTreeClassifier(),
      5     "KNN": KNeighborsClassifier()
      6 }
      8 compare_classifiers(X_train, X_test, y_train, y_test, classifiers)

NameError: name 'LogisticRegression' is not defined
```

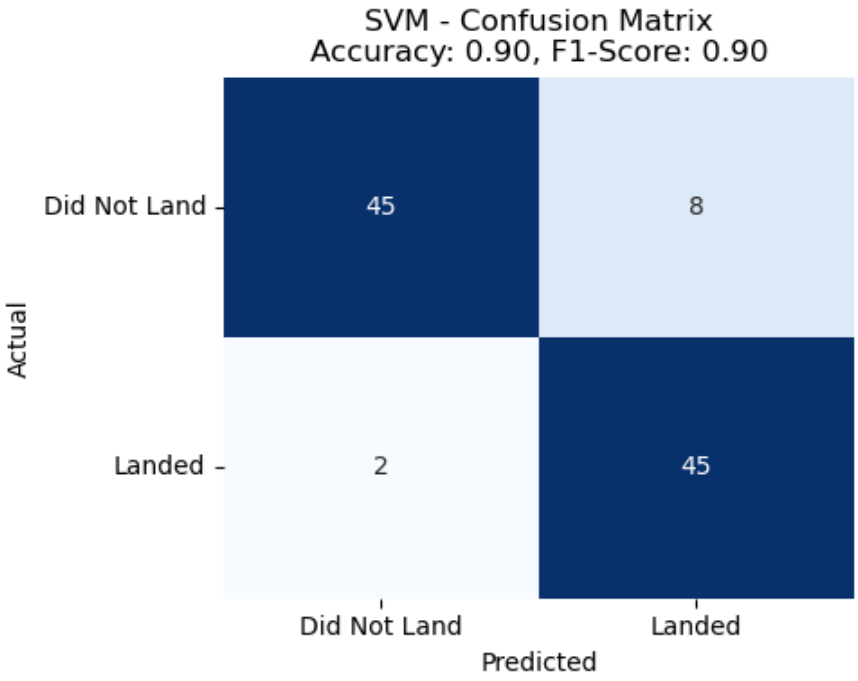
In [3]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
```

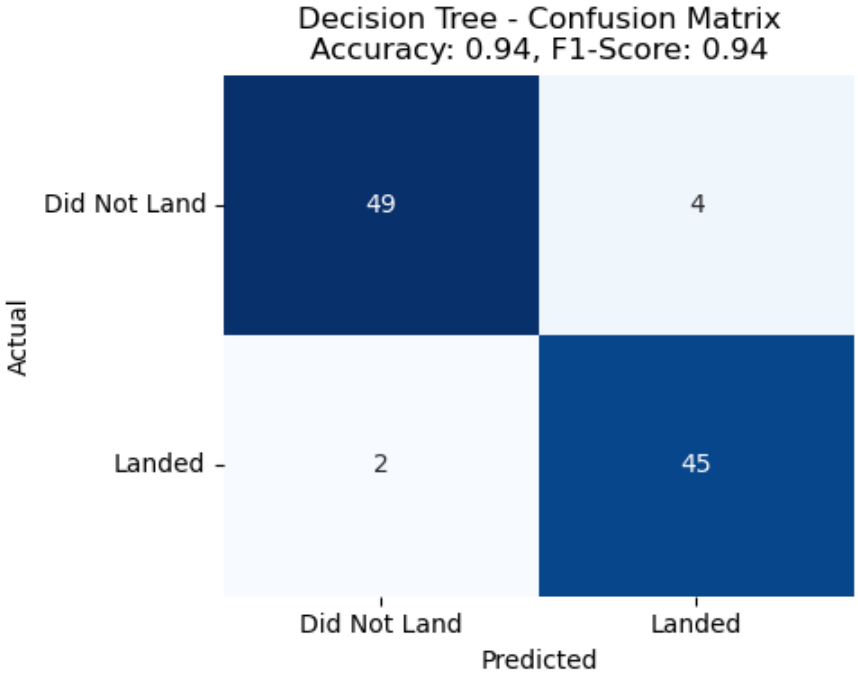
```
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
```

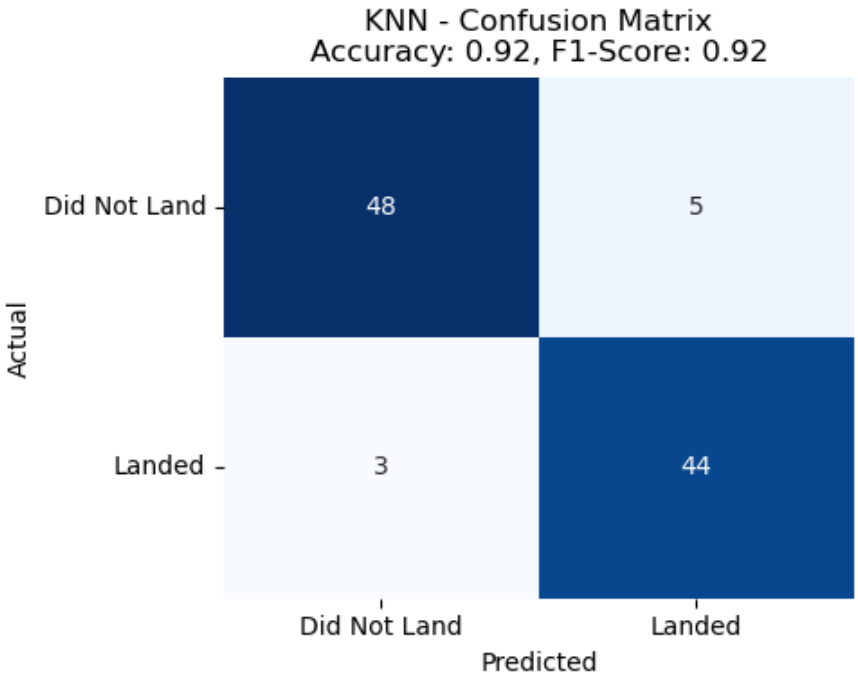
In [39]:
```
classifiers = {
    "Logistic Regression": LogisticRegression(),
    "SVM": SVC(),
    "Decision Tree": DecisionTreeClassifier(),
    "KNN": KNeighborsClassifier()
}
```

In [40]:
```
classifiers = {
    "Logistic Regression": LogisticRegression(),
    "SVM": SVC(),
    "Decision Tree": DecisionTreeClassifier(),
    "KNN": KNeighborsClassifier()
}

compare_classifiers(X_train, X_test, y_train, y_test, classifiers)
```




SVM - Confusion Matrix
Accuracy: 0.90, F1-Score: 0.90
Did Not Land
Landed
Actual
45
8
2
45
Did Not Land
Landed
Predicted


Decision Tree - Confusion Matrix
Accuracy: 0.94, F1-Score: 0.94
Did Not Land
Landed
Actual
49
4
2
45
Did Not Land
Landed
Predicted



In [33]:
```
# Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score, confusion_matrix

# Define confusion matrix plotting function
def plot_confusion_matrix(y_true, y_pred, title='Confusion Matrix'):
    cm = confusion_matrix(y_true, y_pred)
    ax = plt.subplot()
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax)
```

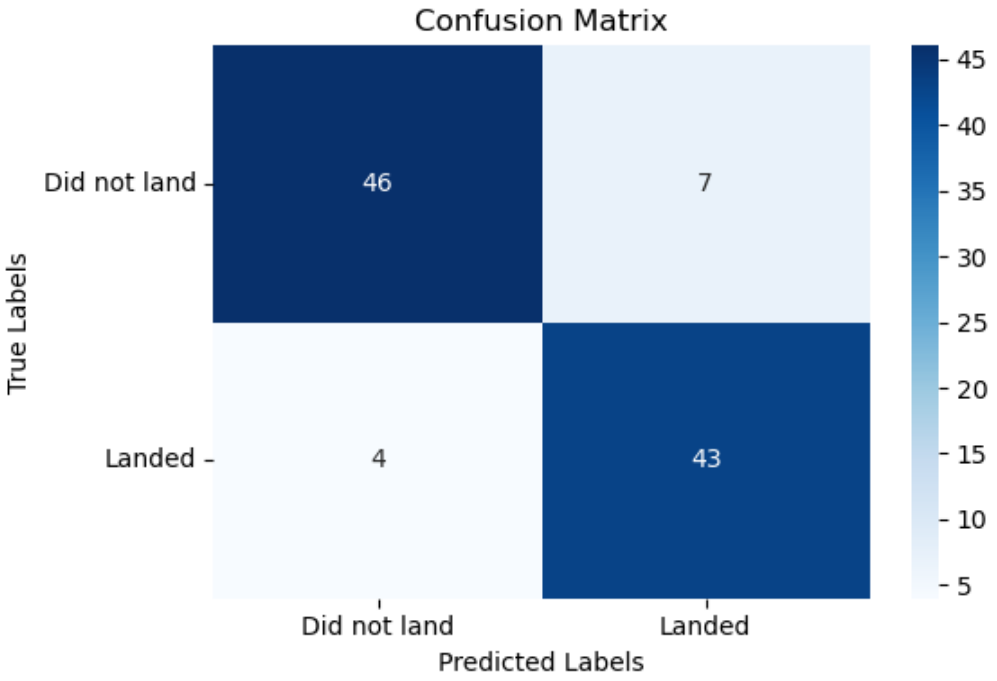
In [34]:
```
%matplotlib inline
```

In [38]:
```
def plot_confusion_matrix(y_true, y_pred, title='Confusion Matrix'):
    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(6, 4))  # Add this for consistent size
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
    plt.xlabel('Predicted Labels')
    plt.ylabel('True Labels')
    plt.title(title)
```

```
    plt.xticks([0.5, 1.5], ['Did not land', 'Landed'])
    plt.yticks([0.5, 1.5], ['Did not land', 'Landed'], rotation=0)
    plt.tight_layout()
    plt.show()  # ✅ Ensures display
```

In [37]:

```
# For Jupyter Notebooks
%matplotlib inline

# Required libraries
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix

# Generate dummy binary classification data
X, y = make_classification(n_samples=500, n_features=5, n_classes=2, random_sta

# Split into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_

# Fit a simple Logistic Regression model
model = LogisticRegression()
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Confusion matrix plotting function
def plot_confusion_matrix(y_true, y_pred, title='Confusion Matrix'):
    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(6, 4))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
    plt.xlabel('Predicted Labels')
    plt.ylabel('True Labels')
    plt.title(title)
    plt.xticks([0.5, 1.5], ['Did not land', 'Landed'])
    plt.yticks([0.5, 1.5], ['Did not land', 'Landed'], rotation=0)
    plt.tight_layout()
    plt.show()

# Plot the matrix
plot_confusion_matrix(y_test, y_pred)
```


Confusion Matrix
Did not land
Landed
True Labels
46
7
4
43
Did not land
Landed
Predicted Labels
45
40
35
30
25
20
15
10
5

In [ ]:

In [ ]: